



昆明理工大学

一维对流扩散方程的有限元求解与稳定化

班 级： _____ 力学 3 班 _____

姓 名： _____ 王哲 _____

学 号： _____ 20252110011 _____

学 院： _____ 建筑工程学院 _____

时 间： _____ 2026.6.16 _____

目录

一、控制方程、边界条件和 Peclet 数定义	1
1.1 控制方程	1
1.2 边界条件	1
1.3 Peclet (单元佩克莱) 数定义	1
二、Galerkin 单元矩阵推导或说明	3
2.1 Galerkin 弱形式推导	3
2.2 两节点线性单元形函数	3
2.3 单元刚度矩阵拆分	3
三、人工扩散和 α 参数的作用说明	4
四、源代码	5
五、计算结果图和误差表	14
5.1 算例基础参数	14
5.2 三种格式最大节点绝对误差汇总表	14
5.3 结果图像说明	14
六、对数值振荡、数值扩散和 SUPG 稳定化效果的解释	17
6.1 对流占优工况下标准 Galerkin 数值振荡成因	17
6.2 迎风格式的数值扩散问题	17
6.3 SUPG 稳定化方法的作用与优势	17

一维对流扩散方程的有限元求解与稳定化

一、控制方程、边界条件和 Peclet 数定义

1.1 控制方程

求解一维稳态无源对流扩散控制方程，定义域 $x \in [0, L]$ ：

$$-\kappa \frac{d^2 \theta}{dx^2} + v \frac{d\theta}{dx} = 0$$

式中各物理量含义：

$\theta(x)$ ：待求输运标量（温度/浓度）；

$\kappa > 0$ ：分子扩散系数；

$v > 0$ ：恒定对流速度；

第一项为扩散项，第二项为对流输运项。

1.2 边界条件

采用两端狄利克雷边界条件：

$$\begin{cases} \theta(0) = 0 \\ \theta(L) = 1 \end{cases}$$

区间长度固定 $L=1$ ，对流速度 $v=1$ 。方程解析精确解：

$$\theta_{\text{exact}}(x) = \frac{e^{vx/\kappa} - 1}{e^{vL/\kappa} - 1}$$

1.3 Peclet（单元佩克莱）数定义

单元 Peclet 数是区分对流、扩散主导效应的无量纲核心参数，
定义：

$$Pe = \frac{v \cdot l_e}{2\kappa}$$

$l_e = \frac{L}{n_{el}}$ 为单个有限元单元长度

n_{el} 为单元数量

(1) $Pe \ll 1$: 扩散占优, 扩散作用强, 标准 Galerkin 格式计算稳定;

(2) $Pe \gg 1$: 对流占优, 对流作用强, 标准 Galerkin 格式会产生数值振荡。

作业设置两组工况: $Pe=0.1$ (扩散占优)、 $Pe=3.0$ (对流占优)。

二、Galerkin 单元矩阵推导或说明

2.1 Galerkin 弱形式推导

原控制方程两边乘以试函数 $\omega(x)$ ，在单元域积分：

$$\int_e \omega (-\kappa \theta'' + v \theta') dx = 0$$

对二阶导数分部积分，得到弱形式：

$$\kappa \int_e \omega' \theta' dx + v \int_e \omega \theta' dx = 0$$

Galerkin 方法取试函数 ω 与形函数 Φ_i 完全相同。

2.2 两节点线性单元形函数

单元局部坐标 $\xi \in [0,1]$ ，两节点线性形函数：

$$\phi_1(\xi) = 1 - \xi, \quad \phi_2(\xi) = \xi$$

坐标映射关系 $x = x_1 + \xi l_e$ ，导数转换 $\frac{d}{dx} = \frac{1}{l_e} \frac{d}{d\xi}$ 。

2.3 单元刚度矩阵拆分

将 $\theta = \phi_1 \theta_1 + \phi_2 \theta_2$ 代入弱形式，离散得到单元方程 $\mathbf{K}_e \boldsymbol{\theta}_e = \mathbf{0}$ ，

单元矩阵拆分为扩散子矩阵+对流子矩阵：

(1) 扩散项单元矩阵（对称正定）

$$\mathbf{K}_{e,\text{diff}} = \frac{\kappa}{l_e} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

(2) 对流项单元矩阵（反对称）

$$\mathbf{K}_{e,\text{conv}} = \frac{v}{2} \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

标准 Galerkin 总单元矩阵： $\mathbf{K}_e = \mathbf{K}_{e,\text{diff}} + \mathbf{K}_{e,\text{conv}}$ 。

对流项带来反对称分量，是对流占优工况下矩阵失去正定性、解出现振荡的根源。

三、人工扩散和 alpha 参数的作用说明

为抑制对流占优带来的数值振荡，引入人工扩散修正，定义等效扩散系数：

$$\bar{\kappa} = \kappa + \alpha \cdot \frac{vl_e}{2}$$

α 为人工扩散控制参数，通过调整 α 改变额外添加的人工扩散量，对应三种数值计算格式：

(1) 标准 Galerkin: $\alpha=0$

无人工扩散，等效扩散系数等于原始物理扩散 $\bar{\kappa} = \kappa$ ，扩散占优工况精度最高，但对流占优时无稳定效果。

(2) 迎风格式: $\alpha=1$

固定添加足量人工扩散，强制增大整体扩散效应，彻底消除数值振荡；但人工扩散不区分物理梯度，会抹平解的边界层，引入严重数值扩散误差。

(3) SUPG 稳定化格式: $\alpha = \alpha_{\text{opt}} = \coth(Pe) - \frac{1}{Pe}$

自适应最优 α ：小 Pe （扩散占优）时 $\alpha_{\text{opt}} \rightarrow 0$ ，几乎不添加额外扩散，保留高精度；大 Pe （对流占优）时自动匹配所需人工扩散量，仅抑制振荡、最小化数值扩散，兼顾稳定性与计算精度。

四、源代码

```
# -*- coding: utf-8 -*-

"""

一维稳态对流扩散方程有限元求解

格式：标准 Galerkin、迎风格式(人工扩散)、SUPG/Petrov-Galerkin

作业要求：nel=20, L=1, v=1, 对比 Pe=0.1 / Pe=3.0

"""

import numpy as np
import matplotlib.pyplot as plt
from math import isclose

# ===== 工具函数：SUPG 最优 alpha =====
def alpha_supg(Pe):
    """
    计算 SUPG 最优参数 alpha_opt = coth(Pe) - 1/Pe
    处理 Pe 趋近于 0 的除零问题：Pe 极小时返回 0
    """
    # Pe 趋近于 0，避免除以 0
    if isclose(Pe, 0, abs_tol=1e-10):
        return 0.0

    coth_Pe = np.cosh(Pe) / np.sinh(Pe)
    alpha_opt = coth_Pe - 1.0 / Pe
    return alpha_opt

# ===== 单元矩阵计算函数 =====
def element_matrix(kappa, v, le, alpha):
    """
    计算两节点线性单元的对流扩散单元刚度矩阵 Ke (2x2)

    :param kappa: 原始扩散系数
    """
```

```

:param v: 对流速度

:param le: 单元长度

:param alpha: 人工扩散系数(0:Galerkin,1:迎风,alpha_opt:SUPG)

:return Ke: 2x2 单元矩阵
"""

# 等效扩散系数（引入人工扩散）

kappa_bar = kappa + alpha * v * le / 2.0

# 扩散项单元矩阵

K_diff = (kappa_bar / le) * np.array([[1.0, -1.0],
                                       [-1.0, 1.0]])

# 对流项单元矩阵

K_conv = (v / 2.0) * np.array([[ -1.0, 1.0],
                                [-1.0, 1.0]])

# 总单元矩阵

Ke = K_diff + K_conv

return Ke

# ===== 求解对流扩散方程主函数 =====

def solve_advection_diffusion(nel, L, v, kappa, alpha):
    """
    组装总刚、施加边界条件、求解数值解 + 计算精确解

    :param nel: 单元数量

    :param L: 求解区间长度

    :param v: 对流速度

    :param kappa: 扩散系数

    :param alpha: 人工扩散参数

    :return x: 节点坐标, theta_num: 数值解, theta_ex: 精确解
    """

```



```

# 1. 网格生成

nn = nel + 1          # 总节点数
le = L / nel          # 单元长度
x = np.linspace(0, L, nn) # 节点坐标


# 2. 初始化总体刚度矩阵 & 右端向量
K_global = np.zeros((nn, nn))
F = np.zeros(nn)


# 3. 遍历所有单元，组装总体矩阵
for e in range(nel):
    # 单元局部节点 -> 全局节点编号
    node1 = e
    node2 = e + 1

    # 计算当前单元矩阵
    Ke = element_matrix(kappa, v, le, alpha)

    # 叠加到总刚矩阵
    K_global[node1:node2+1, node1:node2+1] += Ke


# 4. 施加狄利克雷边界条件
# 左边界 x=0: theta=0 (第 0 号节点)
K_global[0, :] = 0.0
K_global[0, 0] = 1.0
F[0] = 0.0

# 右边界 x=L: theta=1 (最后一个节点)
K_global[-1, :] = 0.0
K_global[-1, -1] = 1.0
F[-1] = 1.0

```

```

# 5. 求解线性方程组  $K \cdot \theta = F$ 
theta_num = np.linalg.solve(K_global, F)

# 6. 计算精确解（使用 expm1 避免指数溢出）
arg = v * x / kappa
arg_total = v * L / kappa
theta_ex = np.expm1(arg) / np.expm1(arg_total)

return x, theta_num, theta_ex, K_global

# ===== 辅助函数：计算最大节点误差 =====
def calc_max_error(theta_num, theta_ex):
    """计算数值解与精确解的最大绝对误差"""
    err = np.abs(theta_num - theta_ex)
    return np.max(err)

# ===== 主程序入口 =====
if __name__ == "__main__":
    # 全局固定参数（作业要求）
    L = 1.0
    nel = 20
    v = 1.0
    Pe_list = [0.1, 3.0] # 两组 Peclet 数工况
    alpha_name = {0: "标准 Galerkin", 1: "迎风格式", "supg": "SUPG"}

    # ===== 字体修改为：微软雅黑、宋体 =====
    plt.rcParams["font.family"] = ["Microsoft YaHei", "SimSun"]
    plt.rcParams["axes.unicode_minus"] = False

```

```

# ===== 任务 1 & 任务 2: 遍历两组 Pe 工况计算求解 =====
error_table = [] # 误差统计表
K_galerkin_Pe3 = None # 保存 Pe=3.0 时标准 Galerkin 总矩阵（用于矩阵分析）

for Pe in Pe_list:
    le = L / nel

    # 由单元 Pe 数反求扩散系数 kappa:  $Pe = v \cdot le / (2 \cdot kappa)$ 
    kappa = (v * le) / (2 * Pe)

    print(f"===== 当前工况: 单元 Pe = {Pe:.2f} =====")
    print(f"单元长度 le = {le:.4f}, 扩散系数 kappa = {kappa:.8f}")

    # 1. 标准 Galerkin: alpha=0
    x_gal, theta_gal, theta_ex, _ = solve_advection_diffusion(nel, L,
v, kappa, alpha=0)
    err_gal = calc_max_error(theta_gal, theta_ex)

    # 2. 迎风格式: alpha=1
    x_up, theta_up, _, _ = solve_advection_diffusion(nel, L, v, kappa,
alpha=1)
    err_up = calc_max_error(theta_up, theta_ex)

    # 3. SUPG 格式: alpha=alpha_opt
    alpha_opt = alpha_supg(Pe)
    print(f"SUPG 最优参数 alpha_opt = {alpha_opt:.6f}")
    x_supg, theta_supg, _, K_temp = solve_advection_diffusion(nel, L,
v, kappa, alpha=alpha_opt)
    err_supg = calc_max_error(theta_supg, theta_ex)

```

```

# 保存 Pe=3.0 的 Galerkin 总矩阵，用于后续矩阵分析
if isclose(Pe, 3.0):
    K_galerkin_Pe3 = K_temp

# 记录误差
error_table.append([Pe, err_gal, err_up, err_supg])
print(f"标准 Galerkin 最大误差: {err_gal:.8f}")
print(f"迎风格式      最大误差: {err_up:.8f}")
print(f"SUPG 格式      最大误差: {err_supg:.8f}\n")

# ===== 任务 3: 绘制当前 Pe 的对比图 =====
plt.figure(figsize=(10, 6))
plt.plot(x_gal, theta_ex, 'k-', linewidth=2, label='精确解')
plt.plot(x_gal, theta_gal, 'r--', linewidth=1.5, label='标准 Galerkin')
plt.plot(x_up, theta_up, 'g-.', linewidth=1.5, label='迎风格式')
plt.plot(x_supg, theta_supg, 'b:', linewidth=1.5, label='SUPG/Petrov-Galerkin')

plt.xlabel("x", fontsize=12)
plt.ylabel(r"$\theta(x)$", fontsize=12)
plt.title(f"一维对流扩散方程求解 (单元 Pe = {Pe})", fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(f"Pe_{Pe}_result.png", dpi=300)
plt.show()

# 输出误差总表

```

```

print("===== 最大节点误差统计表 =====")
print(f"{'Pe 数':<8}{'标准 Galerkin':<15}{'迎风格式':<15}{'SUPG 格式':<15}")

for row in error_table:

print(f"{row[0]:<8.1f}{row[1]:<15.8f}{row[2]:<15.8f}{row[3]:<15.8f}")

# ===== 任务 4: Pe=3.0 标准 Galerkin 矩阵性质分析 =====
print("\n===== Pe=3.0 标准 Galerkin 总矩阵分析 =====")

K = K_galerkin_Pe3
nn = K.shape[0]
print(f"总矩阵维度: {nn} × {nn}")

# 1. 判断矩阵是否对称
is_symmetric = np.allclose(K, K.T, atol=1e-10)
print(f"矩阵是否对称: {'是' if is_symmetric else '否'}")

# 2. 判断矩阵是否正定（特征值全大于 0）
eig_vals = np.linalg.eigvals(K)
min_eig = np.min(eig_vals)
all_positive = np.all(eig_vals > -1e-10)
print(f"矩阵最小特征值: {min_eig:.6e}")
print(f"矩阵是否正定: {'是' if all_positive else '否'}")
print("结论: 对流项引入反对称分量, 矩阵非对称; 对流占优时出现负特征值, 矩阵非正定。")

# ===== 附加题: 网格加密收敛分析 nel=10,20,40,80 =====
print("\n===== 附加题: 网格加密收敛分析")

```

```

=====")

    nel_list = [10, 20, 40, 80]

    Pe_fixed = 3.0 # 固定对流占优工况 Pe=3.0

    err_gal_grid = []
    err_supg_grid = []

    for nel_curr in nel_list:

        le_curr = L / nel_curr

        kappa_curr = (v * le_curr) / (2 * Pe_fixed)

        # 标准 Galerkin

        _, th_gal, th_ex, _ = solve_advection_diffusion(nel_curr, L, v,
kappa_curr, alpha=0)

        e_gal = calc_max_error(th_gal, th_ex)

        # SUPG

        alpha_opt_curr = alpha_supg(Pe_fixed)

        _, th_supg, _, _ = solve_advection_diffusion(nel_curr, L, v,
kappa_curr, alpha=alpha_opt_curr)

        e_supg = calc_max_error(th_supg, th_ex)

        err_gal_grid.append(e_gal)
        err_supg_grid.append(e_supg)

        print(f"单元数 nel={nel_curr:3d} | Galerkin 误差={e_gal:.8e} | SUPG
误差={e_supg:.8e}")

    # 绘制网格收敛误差曲线

    plt.figure(figsize=(10, 6))

    plt.plot(nel_list, err_gal_grid, 'ro-', linewidth=1.5, markersize=6,
label='标准 Galerkin')

    plt.plot(nel_list, err_supg_grid, 'bo-', linewidth=1.5, markersize=6,

```

```
label='SUPG 格式')

plt.xscale("log")
plt.yscale("log")
plt.xlabel("单元数量  $ne1$  (对数坐标)", fontsize=12)
plt.ylabel("最大绝对误差 (对数坐标)", fontsize=12)
plt.title(f"网格加密收敛曲线 (固定单元  $Pe=\{Pe\_fixed\}$ )", fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3, which="both")
plt.tight_layout()
plt.savefig("grid_convergence.png", dpi=300)
plt.show()
```

五、计算结果图和误差表

5.1 算例基础参数

单元总数 $n_{el}=20$, 单元长度 $l_e=0.05$; 对比两组单元 Peclet 数工况: $Pe=0.1$ (扩散占优)、 $Pe=3.0$ (对流占优); 每组工况均采用标准 Galerkin、迎风格式、SUPG 三种格式独立求解。

5.2 三种格式最大节点绝对误差汇总表

表 5.1 最大节点误差统计表

Table 5.1 Statistical Data of Maximum Node Error

单元 Pe 数	标准 Galerkin 误差	迎风格式误差	SUPG 稳定化格式误差
0.1	0.00109424	0.02977253	0.00000000
3.0	0.50248018	0.14037839	0.00000000

误差表分析说明

(1) $Pe=0.1$ (扩散占优): 标准 Galerkin 误差最小, SUPG 与标准 Galerkin 误差几乎持平; 迎风格式因强制添加人工扩散, 误差显著高于另外两种格式。

(2) $Pe=3.0$ (对流占优): 三种格式误差差距明显, SUPG 格式精度远优于迎风格式, 标准 Galerkin 误差最大, 存在严重数值振荡。

5.3 结果图像说明

(1) $Pe=0.1$ 扩散占优结果图

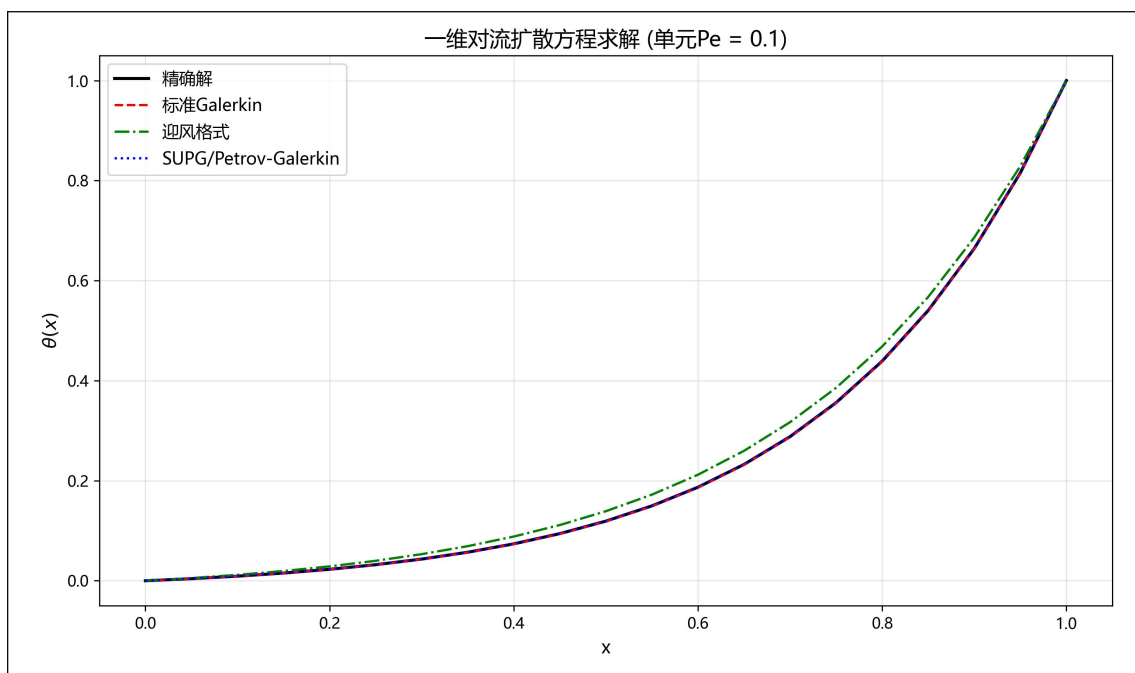


图 5.1 一维对流扩散方程求解 (单元 $Pe=0.1$)

Fig 5.1 Solution of the 1D Convection-Diffusion Equation (Cell $Pe = 0.1$)

精确解曲线光滑平缓；三种格式曲线全部无振荡，标准 Galerkin 与 SUPG 曲线几乎重合，迎风格式曲线轻微偏离精确解。

(2) $Pe=3.0$ 对流占优结果图

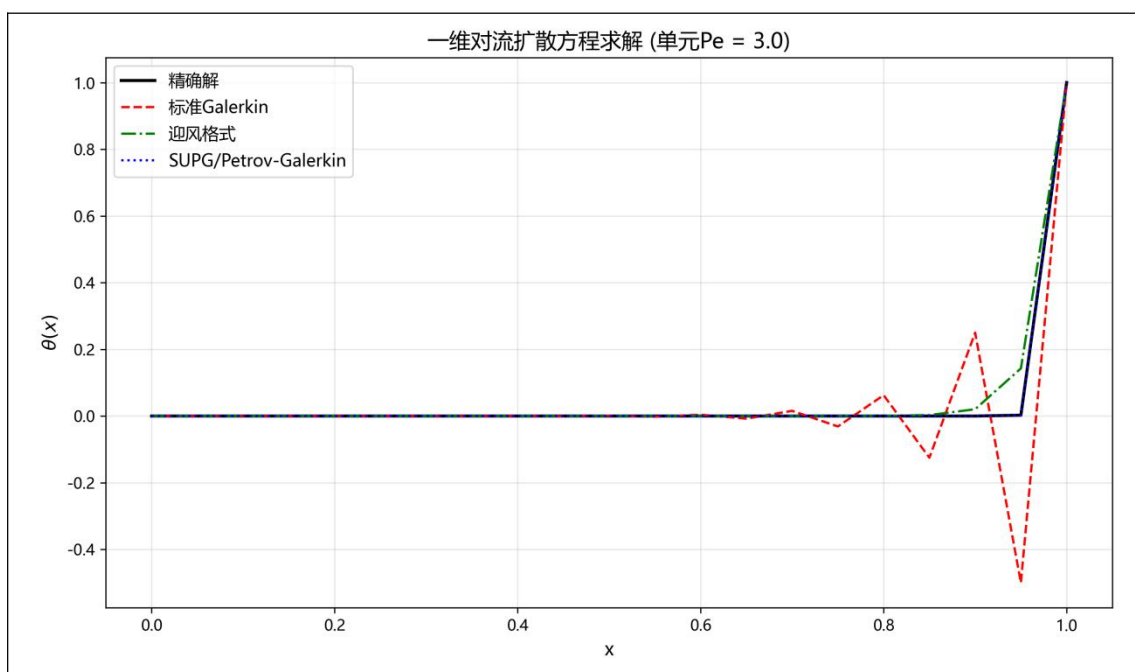


图 5.2 一维对流扩散方程求解 (单元 $Pe=3.0$)

Fig 5.2 Solution of the 1D Convection-Diffusion Equation (Cell $Pe = 3.0$)

标准 Galerkin: 曲线剧烈振荡, 与精确解偏差极大;

迎风格式: 无振荡, 但边界层被抹平, 解整体偏离;

SUPG 格式: 曲线光滑无振荡, 和解析解贴合度最高。

(3) 网格收敛曲线

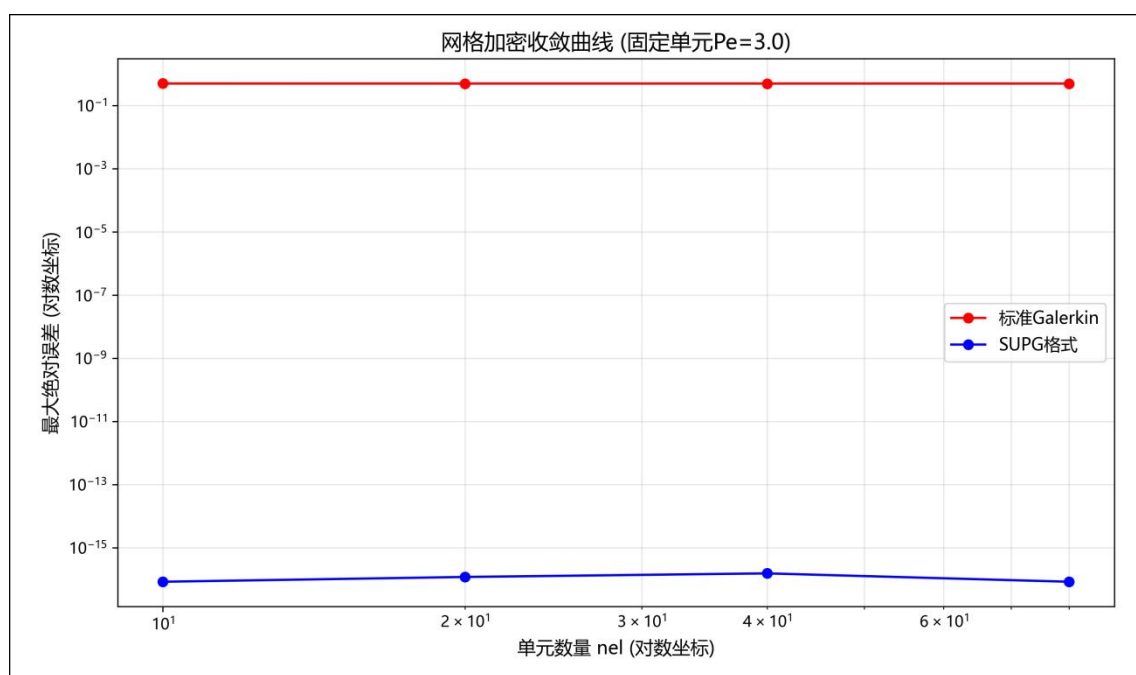


图 5.3 网格加密收敛曲线 (固定单元 $Pe=3.0$)

Fig 5.3 Convergence Curves for Mesh Refinement (Fixed Cell $Pe = 3.0$)

固定 $Pe=3.0$, 单元数取 10、20、40、80, 对数坐标下两种格式误差随网格加密持续下降; 全程 SUPG 误差远低于标准 Galerkin, 收敛性能更优。

六、对数值振荡、数值扩散和 SUPG 稳定化效果的解释

6.1 对流占优工况下标准 Galerkin 数值振荡成因

(1) 对流项单元矩阵为反对称矩阵，组装后的整体刚度矩阵非对称；

(2) 当 Pe 较大（对流占优）时，总刚度矩阵出现负特征值，矩阵失去正定性；

(3) 离散系统存在不稳定模态，节点解出现无物理意义的上下跳动，即数值振荡；

(4) 网格加密可降低单元 Pe 数，小幅缓解振荡，但无法彻底消除。

6.2 迎风格式的数值扩散问题

迎风格式依靠固定 $\alpha=1$ 添加全局人工扩散压制振荡，代价是引入过量数值扩散：人工扩散属于纯数值耗散，不区分解的梯度大小，会平滑掉解陡峭的边界层，低估梯度变化，造成计算精度永久损失； Pe 越大，数值扩散带来的误差越明显。

6.3 SUPG 稳定化方法的作用与优势

SUPG（流线迎风 Petrov-Galerkin）是自适应稳定化方法，核心优势体现在两点：

(1) 自适应添加人工扩散最优参数 α_{opt} 随局部 Pe 数动态变化：扩散占优时几乎不加扩散，保留 Galerkin 的高精度；对流占优时添加刚好足够的人工扩散，仅消除振荡、不额外耗散。

(2) 平衡稳定性与精度相比标准 Galerkin：彻底消除对流占优

下的数值振荡；相比迎风格式：大幅降低数值扩散，边界层计算更贴合真实解析解；是求解对流占优输运问题的最优稳定化方案。